# PROGRAMLAMAYA GİRİŞ: DEĞİŞKENLER, VERİ TİPLERİ, INPUT/OUTPUT

## 1. GİRİŞ

Programlama, bir problemi çözmek için bilgisayara verilen komutlar dizisidir. Bu komutları yazarken verileri saklamak, işlemek ve kullanıcıyla etkileşim kurmak gerekir. İşte bu noktada **değişkenler**, **veri tipleri** ve **input/output (girdi/çıktı)** kavramları devreye girer. Bu bölümde bu üç temel kavram, örneklerle ve karşılaştırmalarla ayrıntılı biçimde ele alınacaktır.

## 2. DEĞİŞKENLER

### 2.1 Değişken Nedir?

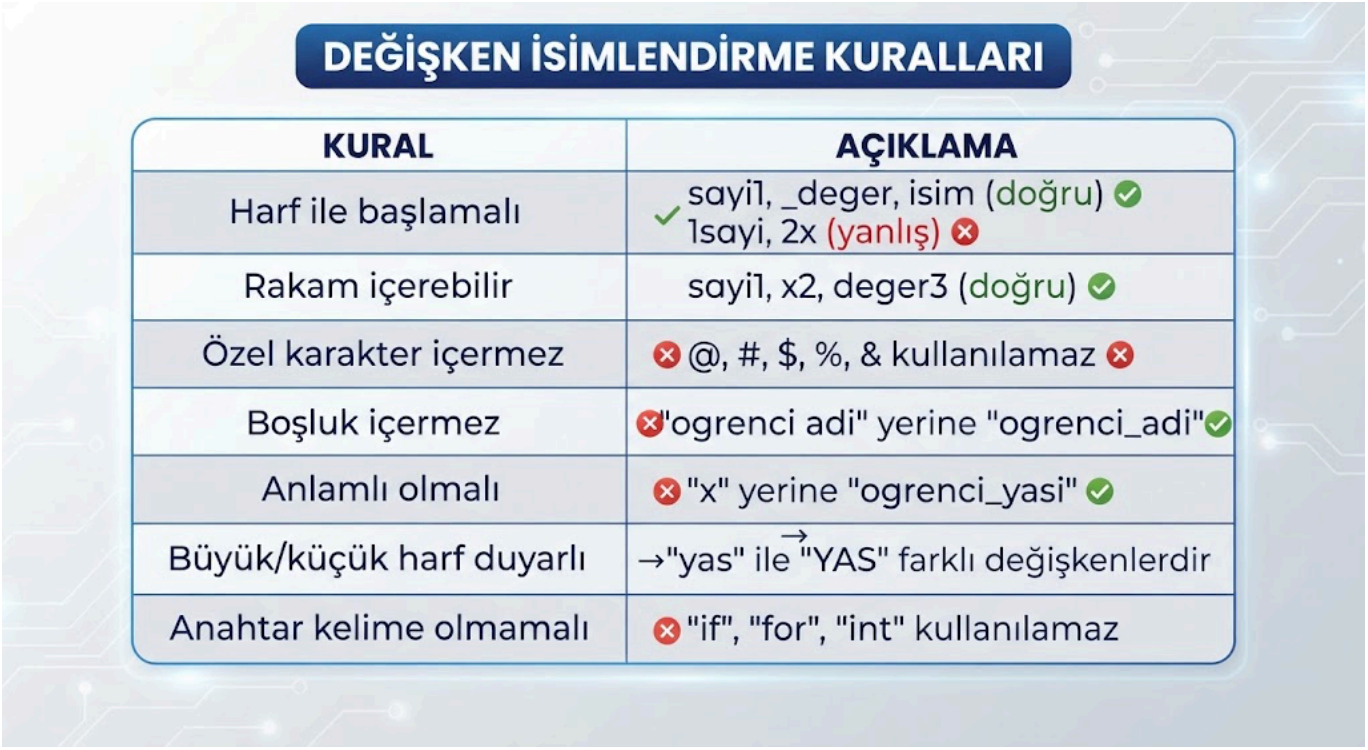
**Değişken**, program çalışırken verilerin geçici olarak saklandığı, isimlendirilmiş bellek hücreleridir. Bir değişken, bir değeri tutar ve program boyunca bu değer değiştirilebilir.




**Günlük hayattan benzetme:** Bir kutu düşünün. Kutunun üzerinde bir etiket (değişken adı) var ve içine bir şey koyuyorsunuz (değer). İstediğiniz zaman içindekini değiştirebilirsiniz.

### 2.2 Değişken Tanımlama Kuralları

Her programlama dilinin kendine özgü kuralları olsa da genel geçer kurallar şunlardır:




Aşağıda **snake_case**, **camelCase** ve C++'ta sık kullanılan diğer isimlendirme stillerini kısa bir tabloyla gösterilmiştir.

---

## İsimlendirme Stilleri (Naming Conventions)

| Stil | Görünüm | Örnek | C++'ta Yaygın Kullanım |
|---|---|---|---|
| **snake_case** | Tüm harfler küçük, kelimeler `_` ile ayrılır | `ogrenci_notu`, `toplam_tutar` | Değişken ve fonksiyon isimlerinde sık kullanılır. |
| **camelCase** | İlk kelime küçük, sonraki kelimelerin ilk harfi büyük | `ogrenciNotu`, `toplamTutar` | Değişken ve fonksiyon isimlerinde sık kullanılır. |
| **PascalCase** | Tüm kelimelerin ilk harfi büyük | `OgrenciNotu`, `ToplamTutar` | Genellikle sınıf (class) ve struct isimlerinde. |
| **UPPER_SNAKE_CASE** | Tüm harfler büyük, kelimeler `_` ile ayrılır | `MAX_DEGER`, `PI_SAYISI` | Sabitler (`const`, `#define`) için kullanılır. |
| **kebab-case** | Kelimeler `-` ile ayrılır | `ogrenci-notu` | C++'ta **kullanılmaz** (`-` operatör olduğu için). |

---

## C++ Örneği

```
const double PI_SAYISI = 3.14159;      // UPPER_SNAKE_CASE (sabit)
int ogrenci_sayisi = 30;               // snake_case
```

```
int ogrenciSayisi = 30;              // camelCase
class OgrenciBilgisi { };            // PascalCase (sınıf)
```

**Not:** C++'ta snake_case ve camelCase sıklıkla tercih edilir; hangisini kullanacağın **takımın veya projenin stil rehberine** bağlıdır. Önemli olan **tutarlı** olmaktır.

## 2.3 Değişken İsimlendirme Örnekleri

**Doğru İsimlendirme:**

```
ogrenci_adi
sayi1
toplam_deger
_gecici
x2
```

**Yanlış İsimlendirme:**

```
1sayi          (rakam ile başlıyor)
ogrenci adi    (boşluk içeriyor)
toplam@deger   (özel karakter içeriyor)
if             (anahtar kelime)
```

## 2.4 Değişken Tanımlama ve Atama

Bir değişken tanımlandığında bellekte yer ayrılır, ardından değer atanır.

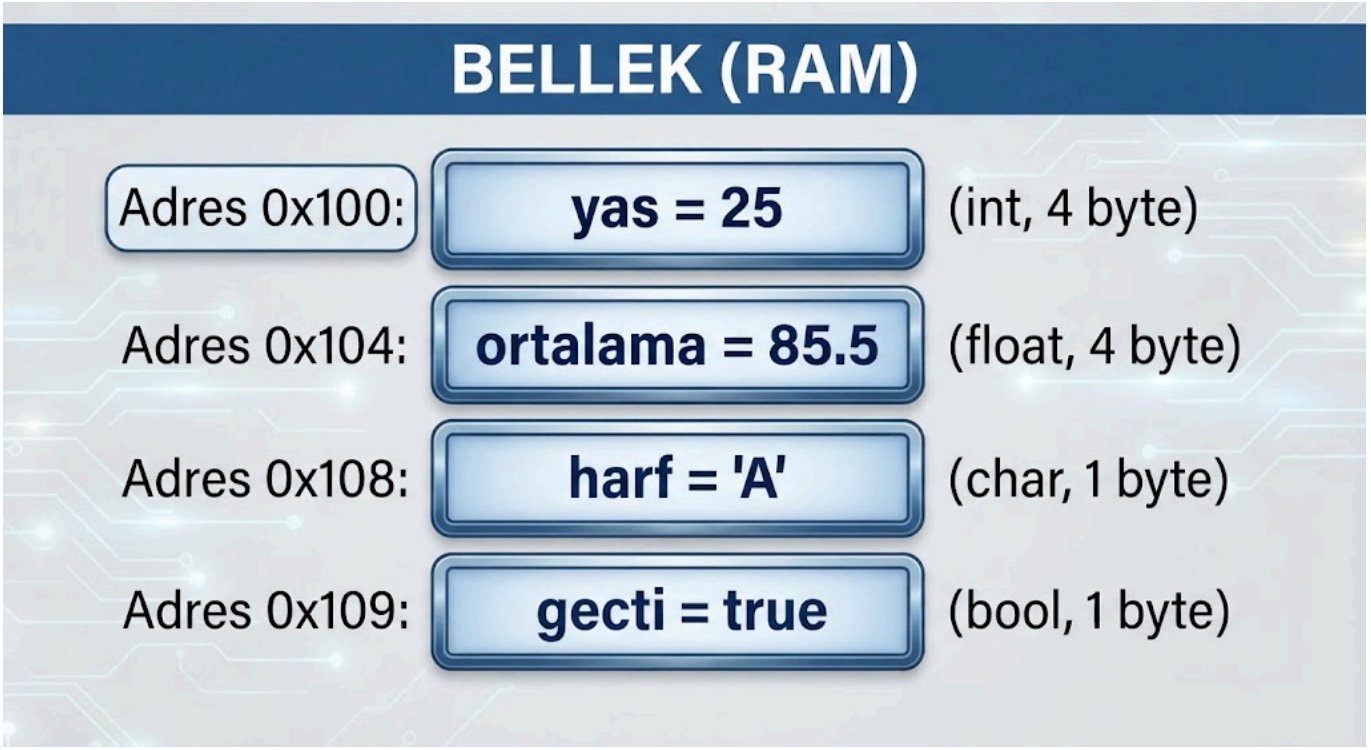
**Genel Yapı:**

```
değişken_tipi değişken_adi = değer;
```

**Örnekler:**

```
int yas = 25;
float ortalama = 85.5;
char harf = 'A';
bool gecti = true;
```

## 2.5 Değişkenlerin Bellekte Temsili




## 2.6 Değişken Türleri (Kapsamına Göre)

```
+----------------------+-----------------------------------------+
| Tür                  | Açıklama                                |
+----------------------+-----------------------------------------+
| Yerel (Local)        | Bir fonksiyon içinde tanımlanır,        |
|                      | yalnızca o fonksiyonda geçerlidir       |
+----------------------+-----------------------------------------+
| Global               | Programın her yerinden erişilebilir     |
+----------------------+-----------------------------------------+
| Statik               | Program boyunca değerini korur          |
+----------------------+-----------------------------------------+
| Sabit (Constant)     | Değeri değiştirilemez                   |
+----------------------+-----------------------------------------+
```

**Örnek:**

```
int global_sayi = 100;        // Global değişken

int main() {
    int yerel_sayi = 50;      // Yerel değişken
    const float PI = 3.14;    // Sabit değişken
    return 0;
}
```

```
# Sabiti dışarıda (global) tanımlıyoruz
PI_SAYISI = 3.14
```

```
def daire_alani(yaricap):
    # Doğrudan erişebiliriz
    return PI_SAYISI * (yaricap ** 2)

def daire_cevresi(yaricap):
    # Doğrudan erişebiliriz
    return 2 * PI_SAYISI * yaricap
```

```
#include <iostream>

// Sabit değeri döndüren fonksiyon
double sabiti_veren_fonksiyon() {
    // const anahtar kelimesi ile değerin değiştirilemez (sabit) olduğunu
belirtiyoruz
    const double VERGI_ORANI = 0.20;
    return VERGI_ORANI;
}

// Hesaplamayı yapan fonksiyon
double hesaplama_yapan_fonksiyon() {
    // Diğer fonksiyonu çağırarak sabit değeri elde ediyoruz
    double oran = sabiti_veren_fonksiyon();
    double fiyat = 100.0 + (100.0 * oran);
    return fiyat;
}

int main() {
    // Sonucu ekrana yazdırmak için test ediyoruz
    std::cout << "Toplam Fiyat: " << hesaplama_yapan_fonksiyon() <<
std::endl;
    return 0;
}
```

```
#include <iostream>

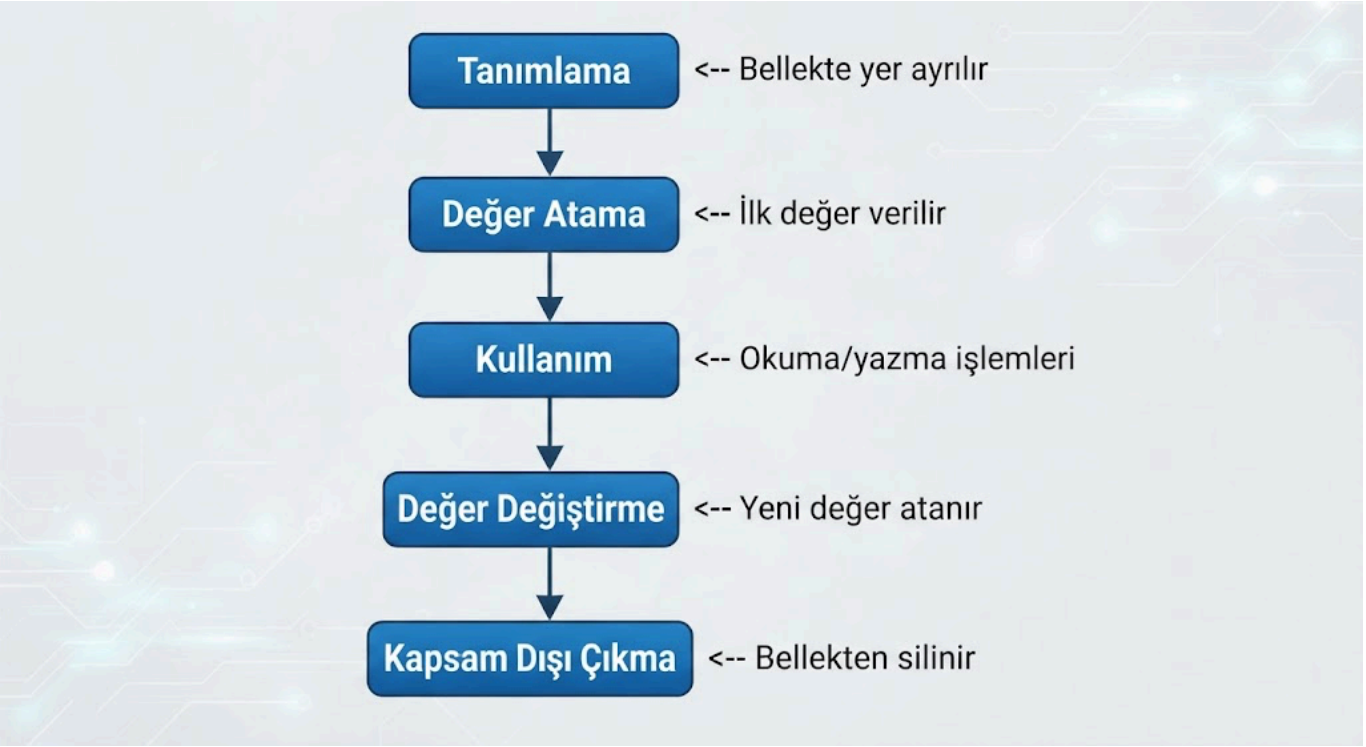
void sayaci_arttir() {
    // Bu satır program boyunca sadece İLK çağrıda çalışır
    static int sayac = 0;

    sayac++;
    std::cout << "Sayacin anlik degeri: " << sayac << std::endl;
}

int main() {
    sayaci_arttir(); // Çıktı: 1
    sayaci_arttir(); // Çıktı: 2
    sayaci_arttir(); // Çıktı: 3
```

```
    return 0;
}
```

### 2.7 Değişkenlerin Yaşam Döngüsü




**Statik local global ve sabitler bellek yönetimi açısından nasıl değerlendirilir?**

---

## 3. VERİ TİPLERİ

### 3.1 Veri Tipi Nedir?

**Veri tipi**, bir değişkenin bellekte ne kadar yer kaplayacağını, hangi türde veri saklayacağını ve hangi işlemlerin yapılabileceğini belirler.

```
+-------------------+
|   VERİ TİPİ       |
|   - Boyut         |
|   - Aralık        |
|   - İşlemler      |
+-------------------+
```

### 3.2 Temel Veri Tipleri

| Veri Tipi | Boyut | Aralık / Sınır | Örnek |
|---|---|---|---|
| **int** | 4 byte (32-bit) | $-2,147,483,648$ ile $2,147,483,647$ | `25`, `-100`, `0` |

| Veri Tipi | Boyut | Aralık / Sınır | Örnek |
|---|---|---|---|
| **float** | 4 byte (32-bit) | $\pm 3.4 \times 10^{-38}$ ile $\pm 3.4 \times 10^{38}$ | `3.14f`, `-0.5f`, `2.0f` |
| **double** | 8 byte (64-bit) | $\pm 1.7 \times 10^{-308}$ ile $\pm 1.7 \times 10^{308}$ | `3.141592653589793` |
| **char** | 1 byte (8-bit) | $-128$ ile $127$ (İşaretli) / $0$ ile $255$ (İşaretsiz) | `'A'`, `'5'`, `'?'` |
| **bool** | 1 byte | `true` / `false` (1 veya 0) | `true`, `false` |
| **string** | Değişken (Dinamik) | Belleğin elverdiği ölçüde karakter dizisi | `"Merhaba"`, `"ABC"` |

Veri Tiplerinin Açıklamaları ve Hassasiyet Detayları

- **int (Integer / Tam Sayı):**
- *Açıklama:* Kesirli kısmı olmayan tam sayıları saklamak için kullanılır. Genellikle döngü sayaçları, yaş, adet gibi hesaplamalarda tercih edilir. 32-bit sistemlerde bu aralıkta değerler alır.
- **float (Single-precision Floating-point / Tek Duyarlıklı Ondalıklı Sayı):**
- *Açıklama:* Ondalıklı sayıları tutar. Virgülden sonraki hassasiyeti **yaklaşık 6 ila 7 basamaktır**. Daha büyük veya daha hassas hesaplamalarda veri kaybı yaşanabilir. Kod yazarken sonuna `f` harfi (`3.14f` gibi) eklenerek float olduğu belirtilir.
- **double (Double-precision Floating-point / Çift Duyarlıklı Ondalıklı Sayı):**
- *Açıklama:* `float` tipine göre iki kat daha fazla bellek (8 byte) kullanır ve virgülden sonraki hassasiyeti **yaklaşık 15 ila 17 basamağa** çıkarır. Bilimsel ve yüksek hassasiyet gerektiren matematiksel hesaplamalarda tercih edilir.
- **char (Character / Karakter):**
- *Açıklama:* Tek bir karakteri (harf, rakam veya sembol) saklar. Aslında arka planda ASCII tablosundaki sayısal karşılığı olan bir tam sayı olarak tutulur (Örn: 'A' karakteri ASCII tablosunda 65'tir). Tek tırnak (`' '`) içinde tanımlanır.
- **bool (Boolean / Mantıksal):**
- *Açıklama:* Sadece iki durumdan birini (`true` doğru/1 veya `false` yanlış/0) saklar. Genellikle `if-else` gibi koşul yapı کینکه llerinde ve mantıksal kontrol mekanizmalarında kararlar almak için kullanılır.
- **string (String / Metin):**
- *Açıklama:* Bir veya birden fazla karakterin birleşmesiyle oluşan metinsel ifadeleri (karakter dizilerini) saklar. Temelde bir ilkel (primitive) veri tipi değildir; birçok dilde nesne (object) veya char dizisi olarak yönetilir. Boyutu içeriğindeki karakter sayısına göre dinamik olarak değişir ve çift tırnak (`" "`) içinde yazılır.
- **void (Boş / Yokluk):**

- *Açıklama:* Değer barındırmayan veya veri tipi olmadığını belirten bir ifadedir. Genellikle fonksiyonların bir değer döndürmeyeceğini belirtmek için (`void fonksiyonAdi()`) veya tür dönüşümlerinde işaretçi (pointer) tiplerinde kullanılır. Doğrudan değişken olarak tanımlanamaz.

### 3.3 Sayısal Veri Tipleri

#### 1. Tam Sayı Tipleri (Integer Types) Tablosu

| Tip | Boyut | Aralık | Kullanım / Açıklama |
|---|---|---|---|
| **short** | 2 byte (16-bit) | $-32,768$ ile $32,767$ | Küçük aralıktaki tamsayılar için bellekten tasarruf amacıyla kullanılır. |
| **int** | 4 byte (32-bit) | $-2,147,483,648$ ile $2,147,483,647$ ($-2^{31} \dots 2^{31}-1$) | Günlük programlamada genel amaçlı kullanılan standart tam sayı tipidir. |
| **long** | 4 veya 8 byte | İşletim sistemine göre değişir (Genelde 32-bit Windows'ta 4, Linux/64-bit sistemlerde 8 byte) | `int` tipine göre daha geniş aralık gerektiren durumlarda kullanılır. |
| **long long** | 8 byte (64-bit) | $-9,223,372,036,854,775,808$ ile $9,223,372,036,854,775,807$ ($-2^{63} \dots 2^{63}-1$) | Çok büyük tam sayıları (örneğin milisaniye cinsinden zaman damgaları vb.) saklamak için kullanılır. |
| **unsigned int** | 4 byte (32-bit) | $0$ ile $4,294,967,295$ ($0 \dots 2^{32}-1$) | Sadece pozitif (negatif olamayan) değer alacak tam sayılar için kullanılır; aralığın pozitif yönünü iki katına çıkarır. |

---

#### 2. Tam Sayı Tiplerinin Detaylı Açıklamaları

- **short (Kısa Tam Sayı):** Küçük bellek alanı kaplaması gereken durumlarda (örneğin büyük diziler veya bellek kısıtlı gömülü sistemler) tercih edilir. 16 bittir.
- **int (Standart Tam Sayı):** En yaygın kullanılan tam sayı tipidir. Çoğu modern sistemde 32-bit (4 byte) uzunluğundadır.
- **long / long long (Uzun Tam Sayılar):** Standart `int` sınırlarının yetmediği büyük finansal hesaplamalar, büyük kimlik numaraları veya astronomik sayılar için tercih edilir. `long long` mimariden bağımsız olarak garantili 64-bit alan sunar.
- **unsigned int (İşaretsiz Tam Sayı):** Başına `unsigned` kelimesi getirilen tipler negatif değer alamaz. Negatif sayılara ihtiyaç duyulmadığında, pozitif yönlü üst sınırı iki katına çıkararak daha büyük pozitif sayılar saklamaya olanak tanır.

---

#### 3. Ondalıklı Sayı Tipleri (Floating-point Types) Tablosu

| Tip | Boyut | Hassasiyet | Kullanım / Açıklama |
|---|---|---|---|
| **float** | 4 byte (32-bit) | ~6-7 anlamlı basamak ($\pm 3.4 \times 10^{-38}$ ile $\pm 3.4 \times 10^{38}$) | Düşük bellek maliyeti gerektiren genel ondalıklı hesaplamalar (örneğin grafik/oyun programlama). |
| **double** | 8 byte (64-bit) | ~15-17 anlamlı basamak ($\pm 1.7 \times 10^{-308}$ ile $\pm 1.7 \times 10^{308}$) | Standart bilimsel ve matematiksel hesaplamalarda kullanılan varsayılan ondalıklı tip. |
| **long double** | 10, 12 veya 16 byte | ~19+ anlamlı basamak (Sisteme göre değişir) | Ekstra yüksek hassasiyet gerektiren ileri düzey bilimsel ve mühendislik hesaplamaları. |

## 4. Ondalıklı Sayı Tiplerinin Detaylı Açıklamaları

- **float (Tek Duyarlıklı):** Ondalıklı sayılar için temel tiptir ancak virgülden sonraki hassasiyeti sınırlıdır. Yüksek hassasiyet gereken yerlerde yuvarlama hatalarına yol açabilir.
- **double (Çift Duyarlıklı):** Bilgisayarlarda ondalıklı sayı yazıldığında (örneğin `3.14`) aksi belirtilmedikçe derleyici bunu otomatik olarak `double` olarak ele alır. `float`'a göre hem aralığı hem de virgülden sonraki hassasiyeti çok daha yüksektir.
- **long double (Genişletilmiş Duyarlıklı):** Donanım mimarisine bağlı olarak (özellikle x86 mimarisinde 80-bit/10-byte veya bazı sistemlerde 128-bit) en yüksek hassasiyeti sunan ondalıklı veri tipidir.

## 3.4 Karakter ve Metin Tipleri

```
+---------------------+-----------------------------------------+
| Tip                 | Açıklama                                |
+---------------------+-----------------------------------------+
| char                | Tek karakter: 'A', '5', '?'             |
| string              | Karakter dizisi: "Merhaba Dünya"        |
| char dizisi         | char isim[20] = "Ali";                  |
+---------------------+-----------------------------------------+
```

**Karakter ve String Farkı:**

```
char harf = 'A';           // Tek karakter, tek tırnak
char isim[] = "Ali";       // Karakter dizisi, çift tırnak
string mesaj = "Merhaba";  // String (C++'ta)
```

## 3.5 Mantıksal Veri Tipi

```
bool dogru = true;     // 1 olarak saklanır
bool yanlis = false;   // 0 olarak saklanır
```

Mantıksal veri tipi genellikle karar yapılarında kullanılır.

```
if (gecti == true) {
    printf("Geçti");
}
```

## 3.6 Veri Tiplerinin Bellekte Temsili




- Little endian, bilgisayar hafızasında çok baytlı (multi-byte) verilerin saklanma düzenidir ve en az anlamlı baytın (en küçük değerin) en düşük bellek adresine yazılması anlamına gelir
- IEEE 754, bilgisayarların ondalıklı ve kesirli sayıları (floating-point) ikili sistemde (binary) nasıl saklayacağını ve işleyeceğini belirleyen uluslararası bir standarttır.
- ASCII (American Standard Code for Information Interchange - Bilgi Değişimi İçin Amerikan Standart Kodlama Sistemi), bilgisayarlardaki harfleri, rakamları ve noktalama işaretlerini sayısal kodlara çeviren bir karakter tablosudur.

## 3.7 Veri Tipi Dönüşümleri

**Otomatik Dönüşüm (Implicit):**

Küçük tipten büyük tipe doğru otomatik dönüşüm yapılır.

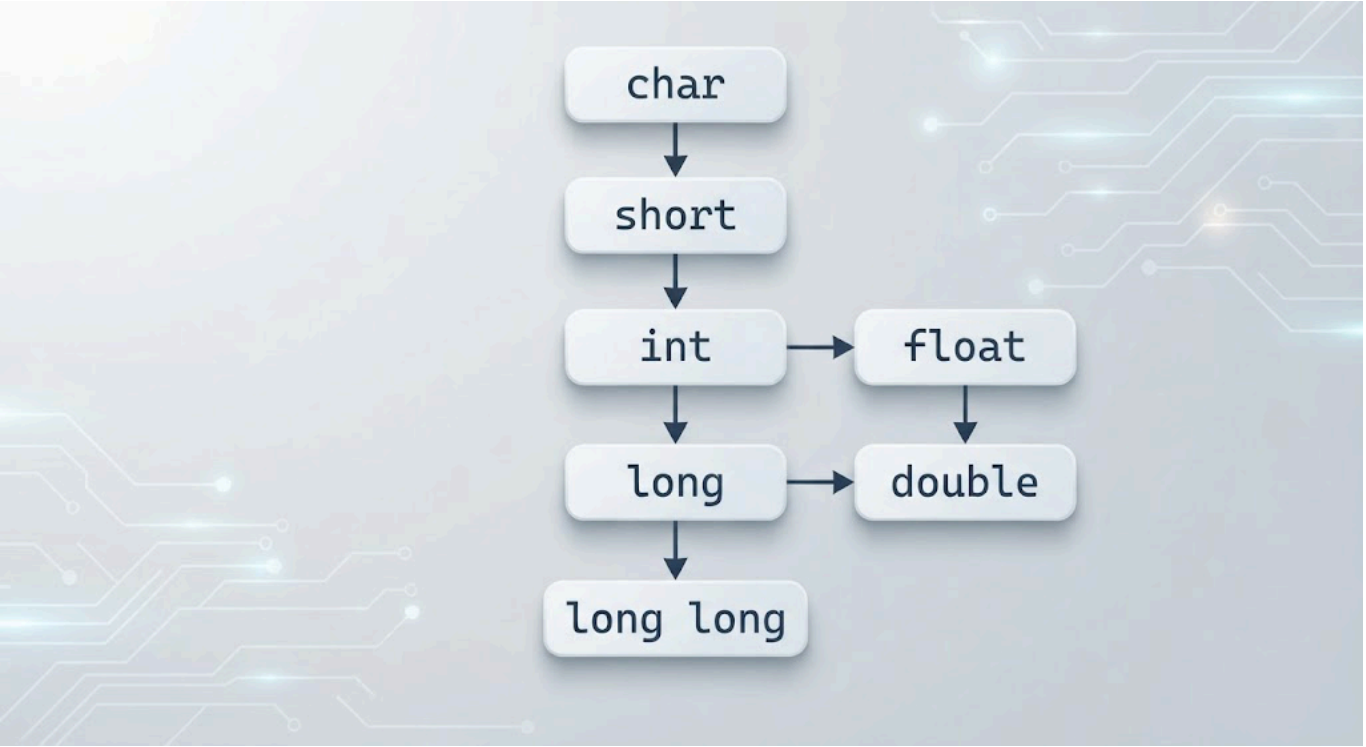
```
int sayi = 10;
float ondalik = sayi;    // int -> float (otomatik)
```

**Zorunlu Dönüşüm (Explicit / Casting):**

Programcı tarafından açıkça belirtilir.

```
float ondalik = 3.14;
int sayi = (int)ondalik;   // float -> int (ondalık kısım kaybolur)
```

**Dönüşüm Şeması:**




**Dönüşüm Örnekleri:**

```
int a = 10;
float b = 3.5;
float sonuc = a + b;      // a otomatik float'a dönüşür

int x = 10;
int y = 3;
float z = x / y;          // 3.0 (tam sayı bölmesi)
float w = (float)x / y;   // 3.333... (ondalıklı bölme)
```

## 3.8 Veri Tipi Seçim Rehberi

```
+----------------------------------+----------------------------------------
--+
| İhtiyaç                          | Önerilen Tip
|
+----------------------------------+----------------------------------------
--+
| Tam sayı (genel)                 | int
```

```
|
| Çok büyük tam sayı                | long long
|
| Negatif olmayan tam sayı          | unsigned int
|
| Ondalıklı sayı (genel)            | float
|
| Yüksek hassasiyetli ondalık       | double
|
| Tek karakter                      | char
|
| Metin                             | string / char dizisi
|
| Doğru/Yanlış                      | bool
|
+-----------------------------------+-------------------------------------
--+
```

---

# 4. INPUT / OUTPUT (GİRDİ / ÇIKTI)

## 4.1 Input (Girdi) Nedir?

**Input**, programın dışarıdan aldığı verilerdir. Kullanıcıdan, dosyadan, sensörden veya başka bir programdan gelebilir.




## 4.2 Output (Çıktı) Nedir?

**Output**, programın işlem sonucunda ürettiği verilerdir. Ekrana, dosyaya, yazıcıya veya başka bir programa gönderilebilir.




## 4.3 Input/Output Akış Şeması




## 4.4 Girdi Yöntemleri

| Yöntem | Açıklama |
|---|---|
| Klavye | Kullanıcıdan manuel giriş |
| Dosya | Dosyadan okuma |
| Sensör | Fiziksel ölçüm verisi |
| Ağ | İnternet üzerinden veri |

```
| Veritabanı           | Kayıtlı verilerin okunması              |
| Komut satırı         | Program çalıştırılırken verilen argüman|
+----------------------+-----------------------------------------+
```

### 4.5 Çıktı Yöntemleri

```
+----------------------+-----------------------------------------+
| Yöntem               | Açıklama                                |
+----------------------+-----------------------------------------+
| Ekran                | Görsel çıktı                            |
| Dosya                | Dosyaya yazma                           |
| Yazıcı               | Fiziksel çıktı                          |
| Ağ                  | Başka bir programa gönderme            |
| Veritabanı           | Kayıt ekleme/güncelleme                 |
| Ses                 | Hoparlör çıktısı                       |
+----------------------+-----------------------------------------+
```

### 4.6 Farklı Dillerde Input/Output

**C Dili:**

```c
#include <stdio.h>

int main() {
    int yas;
    printf("Yaşınızı girin: ");     // Çıktı
    scanf("%d", &yas);              // Girdi
    printf("Yaşınız: %d\n", yas);   // Çıktı
    return 0;
}
```

**C++ Dili:**

```cpp
#include <iostream>
using namespace std;

int main() {
    int yas;
    cout << "Yaşınızı girin: ";     // Çıktı
    cin >> yas;                     // Girdi
    cout << "Yaşınız: " << yas;     // Çıktı
    return 0;
}
```

**Python Dili:**

```
yas = int(input("Yaşınızı girin: "))   # Girdi
print("Yaşınız:", yas)                  # Çıktı
```

**Java Dili:**

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Yaşınızı girin: ");   // Çıktı
        int yas = scanner.nextInt();            // Girdi
        System.out.println("Yaşınız: " + yas);  // Çıktı
    }
}
```

## Uygulama

Aşağıda, veri tiplerini pekiştirmek için **basit örnekler** ve **alıştırmalar** hazırladım. Önce örneklerle mantığı göstereyim, sonra alıştırmaları cevaplarıyla birlikte verelim.

---

### 1. Basit Örnekler (Veri Tipi – Değer Eşleştirmesi)

| Değer / İfade | Uygun Veri Tipi | Neden? |
|---|---|---|
| `25` | `int` | Tam sayı, küçük ve negatif değil ama genel amaçlı tam sayı tipi yeterli. |
| `-100` | `int` | Negatif tam sayı. |
| `3.14f` | `float` | Ondalıklı ve sonunda `f` var; tek duyarlıklı. |
| `3.141592653589793` | `double` | Yüksek hassasiyetli ondalıklı sayı; varsayılan ondalıklı tip. |
| `'A'` | `char` | Tek bir karakter. |
| `'5'` | `char` | Tek karakter (rakam karakteri, sayı değil). |
| `true` | `bool` | Mantıksal doğru değeri. |
| `"Merhaba"` | `string` | Karakter dizisi (metin). |
| `-9,223,372,036,854,775,808` | `long long` | Çok büyük negatif tam sayı; 64-bit gerekir. |
| `4,294,967,295` | `unsigned int` | Pozitif ve 32-bit işaretsiz üst sınır. |

| Değer / İfade | Uygun Veri Tipi | Neden? |
|---|---|---|
| `'?'` | `char` | Tek sembol karakteri. |
| `0` | `int` | Tam sayı sıfır. |
| `2.0f` | `float` | Ondalıklı ama `f` ekli; float. |
| `false` | `bool` | Mantıksal yanlış değeri. |
| `"ABC"` | `string` | Çok karakterli metin. |

## 2. Alıştırmalar

Aşağıdaki değerlerin karşısına **en uygun veri tipini** yazınız.
(Seçenekler: `short`, `int`, `long long`, `unsigned int`, `float`, `double`, `long double`, `char`, `bool`, `string`)

| # | Değer / İfade | Veri Tipi |
|---|---|---|
| 1 | `42` | ? |
| 2 | `-32,768` | ? |
| 3 | `3.14` | ? |
| 4 | `'Z'` | ? |
| 5 | `false` | ? |
| 6 | `"Programlama"` | ? |
| 7 | `9,223,372,036,854,775,807` | ? |
| 8 | `0` | ? |
| 9 | `-0.5f` | ? |
| 10 | `4,294,967,295` | ? |
| 11 | `'7'` | ? |
| 12 | `1.7e308` | ? |
| 13 | `true` | ? |
| 14 | `-2,147,483,648` | ? |
| 15 | `"A"` | ? |

## 3. Alıştırma Cevapları

| # | Değer / İfade | Veri Tipi | Kısa Açıklama |
|---|---|---|---|
| 1 | `42` | `int` | Küçük tam sayı. |
| 2 | `-32,768` | `short` | 16-bit işaretli alt sınır. |
| 3 | `3.14` | `double` | Varsayılan ondalıklı tip. |
| 4 | `'Z'` | `char` | Tek karakter. |
| 5 | `false` | `bool` | Mantıksal yanlış. |
| 6 | `"Programlama"` | `string` | Karakter dizisi. |
| 7 | `9,223,372,036,854,775,807` | `long long` | 64-bit işaretli üst sınır. |
| 8 | `0` | `int` | Tam sayı sıfır. |
| 9 | `-0.5f` | `float` | `f` ekli negatif ondalık. |
| 10 | `4,294,967,295` | `unsigned int` | 32-bit işaretsiz üst sınır. |
| 11 | `'7'` | `char` | Rakam karakteri. |
| 12 | `1.7e308` | `double` | double üst sınırına yakın değer. |
| 13 | `true` | `bool` | Mantıksal doğru. |
| 14 | `-2,147,483,648` | `int` | 32-bit işaretli alt sınır. |
| 15 | `"A"` | `string` | Çift tırnaklı tek karakter = metin. |

---

Aşağıda C++ diline yönelik **doğru ve yanlış isimlendirme (identifier) örnekleri**, kurallar ve ardından **alıştırmalar + cevaplar** yer almaktadır.

---

## 1. İsimlendirme Kuralları (Özet)

Bir değişken, fonksiyon, sınıf veya sabit ismi (identifier) için:

1. Harf (a-z, A-Z), rakam (0-9) ve alt çizgi (`_`) kullanılabilir.
2. **Rakamla başlayamaz.**
3. **Boşluk içeremez.**
4. **Anahtar kelimeler (keyword) kullanılamaz** (`int`, `for`, `class`, `return` vb.).
5. **Özel karakterler kullanılamaz** (`@`, `#`, `$`, `-`, `.`, `!` vb.).
6. Büyük-küçük harf duyarlıdır (`yas` ile `Yas` farklıdır).
7. Anlamlı ve okunabilir olmalıdır.

---

## 2. Doğru ve Yanlış İsimlendirme Örnekleri

| İsim | Durum | Neden? |
|---|---|---|
| `yas` | ✅ Doğru | Harfle başlıyor, anlamlı. |

| İsim | Durum | Neden? |
|---|---|---|
| `kullaniciAdi` | ✅ Doğru | camelCase, anlamlı. |
| `ogrenci_notu` | ✅ Doğru | snake_case, anlamlı. |
| `sayi1` | ✅ Doğru | Rakam sonda olabilir. |
| `_gecici` | ✅ Doğru | Alt çizgiyle başlayabilir (ama globalde önerilmez). |
| `MAX_DEGER` | ✅ Doğru | Sabit için büyük harf + alt çizgi. |
| `1sayi` | ❌ Yanlış | Rakamla başlayamaz. |
| `kullanici adi` | ❌ Yanlış | Boşluk içeremez. |
| `int` | ❌ Yanlış | Anahtar kelime. |
| `for` | ❌ Yanlış | Anahtar kelime. |
| `class` | ❌ Yanlış | Anahtar kelime. |
| `fiyat$` | ❌ Yanlış | `$` özel karakter. |
| `toplam-tutar` | ❌ Yanlış | `-` özel karakter (çıkarma operatörü). |
| `ogrenci.no` | ❌ Yanlış | `.` özel karakter (üye erişim operatörü). |
| `ad@soyad` | ❌ Yanlış | `@` özel karakter. |
| `#sayi` | ❌ Yanlış | `#` özel karakter (önişlemci). |
| `my variable` | ❌ Yanlış | Boşluk içeriyor. |
| `double` | ❌ Yanlış | Anahtar kelime. |
| `9lives` | ❌ Yanlış | Rakamla başlıyor. |

---

## 3. Alıştırma

Aşağıdaki isimlerin C++ için **geçerli (doğru)** mi yoksa **geçersiz (yanlış)** mi olduğunu işaretleyin. Geçersizse nedenini kısa bir şekilde yazın.

| # | İsim | Doğru / Yanlış | Neden? |
|---|---|---|---|
| 1 | `ogrenciYasi` | ? | ? |
| 2 | `2kere` | ? | ? |
| 3 | `int` | ? | ? |
| 4 | `toplam_tutar` | ? | ? |
| 5 | `fiyat$` | ? | ? |
| 6 | `_sayac` | ? | ? |
| 7 | `kullanici adi` | ? | ? |

| # | İsim | Doğru / Yanlış | Neden? |
|---|---|---|---|
| 8 | `MAX_HIZ` | ? | ? |
| 9 | `return` | ? | ? |
| 10 | `not1` | ? | ? |
| 11 | `veri.1` | ? | ? |
| 12 | `x-y` | ? | ? |
| 13 | `Yas` | ? | ? |
| 14 | `float` | ? | ? |
| 15 | `gecici_deger_1` | ? | ? |
| 16 | `#sayi` | ? | ? |
| 17 | `veri@noktasi` | ? | ? |
| 18 | `sayi_2` | ? | ? |
| 19 | `class` | ? | ? |
| 20 | `Ad Soyad` | ? | ? |

---

## 4. Alıştırma Cevapları

| # | İsim | Doğru / Yanlış | Neden? |
|---|---|---|---|
| 1 | `ogrenciYasi` | ✅ Doğru | Harfle başlıyor, camelCase, anlamlı. |
| 2 | `2kere` | ❌ Yanlış | Rakamla başlayamaz. |
| 3 | `int` | ❌ Yanlış | Anahtar kelime. |
| 4 | `toplam_tutar` | ✅ Doğru | snake_case, anlamlı. |
| 5 | `fiyat$` | ❌ Yanlış | `$` özel karakter. |
| 6 | `_sayac` | ✅ Doğru | Alt çizgiyle başlayabilir (yerel kapsamda uygun). |
| 7 | `kullanici adi` | ❌ Yanlış | Boşluk içeriyor. |
| 8 | `MAX_HIZ` | ✅ Doğru | Sabit ismi için uygun (büyük harf + alt çizgi). |
| 9 | `return` | ❌ Yanlış | Anahtar kelime. |
| 10 | `not1` | ✅ Doğru | Harfle başlıyor, rakam sonda. |
| 11 | `veri.1` | ❌ Yanlış | `.` özel karakter ve rakamla başlıyor. |
| 12 | `x-y` | ❌ Yanlış | `-` özel karakter (operatör). |
| 13 | `Yas` | ✅ Doğru | Harfle başlıyor (büyük harf de olabilir, ama stil farkı). |
| 14 | `float` | ❌ Yanlış | Anahtar kelime. |

| # | İsim | Doğru / Yanlış | Neden? |
|---|---|---|---|
| 15 | `gecici_deger_1` | ✅ Doğru | snake_case, rakam sonda. |
| 16 | `#sayi` | ❌ Yanlış | `#` özel karakter (önişlemci). |
| 17 | `veri@noktasi` | ❌ Yanlış | `@` özel karakter. |
| 18 | `sayi_2` | ✅ Doğru | Harfle başlıyor, rakam sonda. |
| 19 | `class` | ❌ Yanlış | Anahtar kelime. |
| 20 | `Ad Soyad` | ❌ Yanlış | Boşluk içeriyor. |

## 5. Kodda Doğru İsimlendirme Örneği

```cpp
#include <iostream>
using namespace std;

const double PI = 3.14159;          // Sabit: **** büyük harf *****
int ogrenciSayisi = 30;             // camelCase
double ortalamaNot = 85.5;          // camelCase
char harfNotu = 'A';                // tek karakter
bool gectiMi = true;                // mantıksal

int main() {
    int sayi1 = 10;                 // doğru
    int sayi_2 = 20;                // doğru
    // int 1sayi = 5;               // YANLIŞ: rakamla başlıyor
    // int int = 5;                 // YANLIŞ: anahtar kelime
    // int toplam tutar = 5;        // YANLIŞ: boşluk

    cout << "Ogrenci sayisi: " << ogrenciSayisi << endl;
    return 0;
}
```

# Mantık Operatörleri

## Giriş

Mantık operatörleri, iki veya daha fazla durumu karşılaştırıp tek bir sonuç üreten araçlardır. Bu sonuç iki değerden biri olabilir:

- **Doğru (true / 1)**
- **Yanlış (false / 0)**

Günlük hayatta sürekli bunları kullanırız. Örneğin "Hem param var hem vaktim var, o zaman sinemaya giderim" derken aslında AND operatörünü kullanıyoruz. Bu notta önce en temel durumları, sonra daha özel olanları göreceğiz. Kod örnekleri en sonda toplu halde verilecektir.

# Bölüm 1: Temel Mantık Durumları

## 1.1 AND (VE) Operatörü

**Sembol:** &&

**Kural:** İki taraf da doğruysa sonuç doğrudur. Taraflardan biri bile yanlışsa sonuç yanlıştır.

**Günlük hayat örneği:** "Dışarı çıkmak için hem param hem vaktim olmalı."

- Para var + Vakit var → Çıkabilirim (Doğru)
- Para var + Vakit yok → Çıkamam (Yanlış)
- Para yok + Vakit var → Çıkamam (Yanlış)
- Para yok + Vakit yok → Çıkamam (Yanlış)

**Doğruluk Tablosu:**

| A | B | A && B |
|---|---|---|
| Doğru | Doğru | Doğru |
| Doğru | Yanlış | Yanlış |
| Yanlış | Doğru | Yanlış |
| Yanlış | Yanlış | Yanlış |

**Özet:** AND, "hepsi şart" demektir. Tüm koşullar sağlanmalıdır.

---

## 1.2 OR (VEYA) Operatörü

**Sembol:** | |

**Kural:** İki taraftan en az biri doğruysa sonuç doğrudur. İkisi de yanlışsa sonuç yanlıştır.

**Günlük hayat örneği:** "Eve girmek için ya anahtarım ya kartım olmalı."

- Anahtar var + Kart yok → Girebilirim (Doğru)
- Anahtar yok + Kart var → Girebilirim (Doğru)
- Anahtar var + Kart var → Girebilirim (Doğru)
- Anahtar yok + Kart yok → Giremem (Yanlış)

**Doğruluk Tablosu:**

| A | B | A \|\| B |
|---|---|---|
| Doğru | Doğru | Doğru |
| Doğru | Yanlış | Doğru |
| Yanlış | Doğru | Doğru |
| Yanlış | Yanlış | Yanlış |

**Özet:** OR, "biri yeter" demektir. Tek bir koşul sağlansa bile sonuç doğrudur.

---

### 1.3 NOT (DEĞİL) Operatörü

**Sembol:** !

**Kural:** Doğruyu yanlışa, yanlışı doğruya çevirir. Tek başına çalışır, karşılaştırma yapmaz.

**Günlük hayat örneği:** "Hava yağmurlu değilse dışarı çıkarım."

- Yağmur var → Çıkmam (Yanlış)
- Yağmur yok → Çıkarım (Doğru)

**Doğruluk Tablosu:**

| A | !A |
|---|---|
| Doğru | Yanlış |
| Yanlış | Doğru |

**Özet:** NOT, "tersini al" demektir.

---

## Bölüm 2: Özel Mantık Durumu

### 2.1 XOR (ÖZEL VEYA) Operatörü

**Sembol:** ^

**Kural:** İki taraf farklıysa sonuç doğrudur. İki taraf aynıysa (ikisi de doğru ya da ikisi de yanlış) sonuç yanlıştır. Yani "sadece biri doğruysa" doğrudur.

XOR, OR'a benzer gibi görünür ama kritik bir farkı vardır: OR'da ikisi birden doğruyken sonuç doğrudur; XOR'da ise ikisi birden doğruyken sonuç yanlıştır.

**Günlük hayat örneği:** "Bir restoranda ya çorba ya salata alabilirsin. İkisini birden alamazsın, ikisini de almazsan aç kalırsın."

- Çorba var + Salata yok → Tamam (Doğru)
- Çorba yok + Salata var → Tamam (Doğru)
- Çorba var + Salata var → Olmaz (Yanlış)
- Çorba yok + Salata yok → Olmaz (Yanlış)

**Doğruluk Tablosu:**

| A | B | A ^ B |
|---|---|---|
| Doğru | Doğru | Yanlış |
| Doğru | Yanlış | Doğru |
| Yanlış | Doğru | Doğru |

| A | B | A ^ B |
|---|---|---|
| Yanlış | Yanlış | Yanlış |

**Özet:** XOR, "farklı olsun" demektir. Sadece taraflar birbirinden farklıysa doğrudur.

---

## Bölüm 3: Tüm Operatörlerin Karşılaştırması

| A | B | A && B | A \|\| B | A ^ B | !A |
|---|---|---|---|---|---|
| Doğru | Doğru | Doğru | Doğru | Yanlış | Yanlış |
| Doğru | Yanlış | Yanlış | Doğru | Doğru | Yanlış |
| Yanlış | Doğru | Yanlış | Doğru | Doğru | Doğru |
| Yanlış | Yanlış | Yanlış | Yanlış | Yanlış | Doğru |

Bu tabloyu ezberlemek yerine mantığını anlamak yeterlidir:

- **AND:** Hepsi doğruysa doğru.
- **OR:** En az biri doğruysa doğru.
- **NOT:** Tersini alır.
- **XOR:** Sadece biri doğruysa doğru.

| Operatör | Adı | Ne zaman doğru olur |
|---|---|---|
| `&&` | AND (VE) | Tum kosullar dogruysa |
| `\|\|` | OR (VEYA) | En az bir kosul dogruysa |
| `!` | NOT (DEĞİL) | Degeri tersine cevirir |
| `^` | XOR (ÖZEL VEYA) | Sadece bir kosul dogruysa |

---

## Bölüm 4: C++ Kod Örnekleri

Bu bölümde yukarıda anlatılan kavramların C++ dilindeki karşılıkları verilmiştir. C++'ta `true` ekrana `1`, `false` ekrana `0` olarak yazılır.

### 4.1 AND Örneği

```cpp
#include <iostream>
using namespace std;

int main() {
    bool paraVar = true;
    bool vakitVar = false;

    if (paraVar && vakitVar) {
        cout << "Disari cikabilirim" << endl;
```

```
    } else {
        cout << "Cikamam" << endl;
    }
    return 0;
}
```

Cikti:

```
Cikamam
```

Aciklama: Iki kosuldan biri (vakitVar) yanlis oldugu icin AND sonucu yanlistir.

---

## 4.2 OR Örneği

```
#include <iostream>
using namespace std;

int main() {
    bool anahtar = false;
    bool kart = true;

    if (anahtar || kart) {
        cout << "Eve girebilirim" << endl;
    } else {
        cout << "Girememem" << endl;
    }
    return 0;
}
```

Cikti:

```
Eve girebilirim
```

Aciklama: Kosullardan biri (kart) dogru oldugu icin OR sonucu dogrudur.

---

## 4.3 NOT Örneği

```
#include <iostream>
using namespace std;

int main() {
    bool yagmurVar = false;
```

```
    if (!yagmurVar) {
        cout << "Disari cikarim" << endl;
    }
    return 0;
}
```

Cikti:

```
Disari cikarim
```

Aciklama: yagmurVar yanlis oldugu icin !yagmurVar dogru olur ve blok calisir.

---

## 4.4 XOR Örneği

```
#include <iostream>
using namespace std;

int main() {
    bool corba = true;
    bool salata = true;

    if (corba ^ salata) {
        cout << "Siparis tamam" << endl;
    } else {
        cout << "Hatali siparis" << endl;
    }
    return 0;
}
```

Cikti:

```
Hatali siparis
```

Aciklama: Iki taraf da dogru oldugu icin XOR sonucu yanlistir.

---

## 4.5 Tüm Operatörlerin Birlikte Gösterimi

```
#include <iostream>
using namespace std;

int main() {
    bool a = true;
    bool b = false;
```

```
    cout << "a = " << a << ", b = " << b << endl;
    cout << "----------------------" << endl;
    cout << "a && b (AND) = " << (a && b) << endl;
    cout << "a || b (OR)  = " << (a || b) << endl;
    cout << "a ^  b (XOR) = " << (a ^ b)  << endl;
    cout << "!a     (NOT) = " << (!a)     << endl;
    cout << "!b     (NOT) = " << (!b)     << endl;

    return 0;
}
```

Cikti:

```
a = 1, b = 0
----------------------
a && b (AND) = 0
a || b (OR)  = 1
a ^  b (XOR) = 1
!a     (NOT) = 0
!b     (NOT) = 1
```

---

## 4.6 Gerçek Hayat Uygulaması

Bir kapi acma sistemi dusunelim. Kapi acilmasi icin su sartlar gerekli:

1. Anahtar olmali (AND)
2. Ya kirmizi ya mavi kart olmali, ikisi birden olmamali (XOR)
3. Kapi kilitli olmamali (NOT)

```
#include <iostream>
using namespace std;

int main() {
    bool anahtarVar = true;
    bool kirmiziKart = true;
    bool maviKart = false;
    bool kapiKilitli = false;

    bool kapiAcilir = anahtarVar && (kirmiziKart ^ maviKart) &&
!kapiKilitli;

    if (kapiAcilir) {
        cout << "Kapi acildi" << endl;
    } else {
        cout << "Kapi acilmadi" << endl;
    }
```

```
    return 0;
}
```

Cikti:

```
Kapi acildi
```

Adim adim cozum:

1. `anahtarVar` = true
2. `kirmiziKart ^ maviKart` = true ^ false = true
3. `!kapiKilitli` = !false = true
4. `true && true && true` = true

---

Hatirlatma:

- AND → "hepsi sart"
- OR → "biri yeter"
- NOT → "tersini al"
- XOR → "farkli olsun"

## 4.7 Biçimlendirilmiş Çıktı

Çıktıyı düzenlemek için biçimlendirme karakterleri kullanılır.

**C Dilinde Format Belirleyiciler:**

```
+----------------------+-----------------------------------------+
| Belirleyici          | Açıklama                                |
+----------------------+-----------------------------------------+
| %d                   | Tam sayı                                |
| %f                   | Ondalıklı sayı                          |
| %c                   | Karakter                                |
| %s                   | String                                  |
| %lf                  | double                                  |
| %x                   | Onaltılık (hexadecimal)                 |
| %%                   | Yüzde işareti                           |
+----------------------+-----------------------------------------+
```

**Örnek:**

```
int yas = 25;
float boy = 1.75;
char harf = 'A';
char isim[] = "Ali";
```

```
printf("Yaş: %d\n", yas);
printf("Boy: %.2f\n", boy);      // 2 ondalık basamak
printf("Harf: %c\n", harf);
printf("İsim: %s\n", isim);
```

**Çıktı:**

```
Yaş: 25
Boy: 1.75
Harf: A
İsim: Ali
```

## 4.8 Kaçış Karakterleri (Escape Sequences)

| Karakter | Anlamı |
|---|---|
| \n | Yeni satır |
| \t | Sekme |
| \\ | Ters eğik çizgi |
| \" | Çift tırnak |
| \' | Tek tırnak |
| \0 | Null karakter |
| \a | Zil sesi |

**Örnek:**

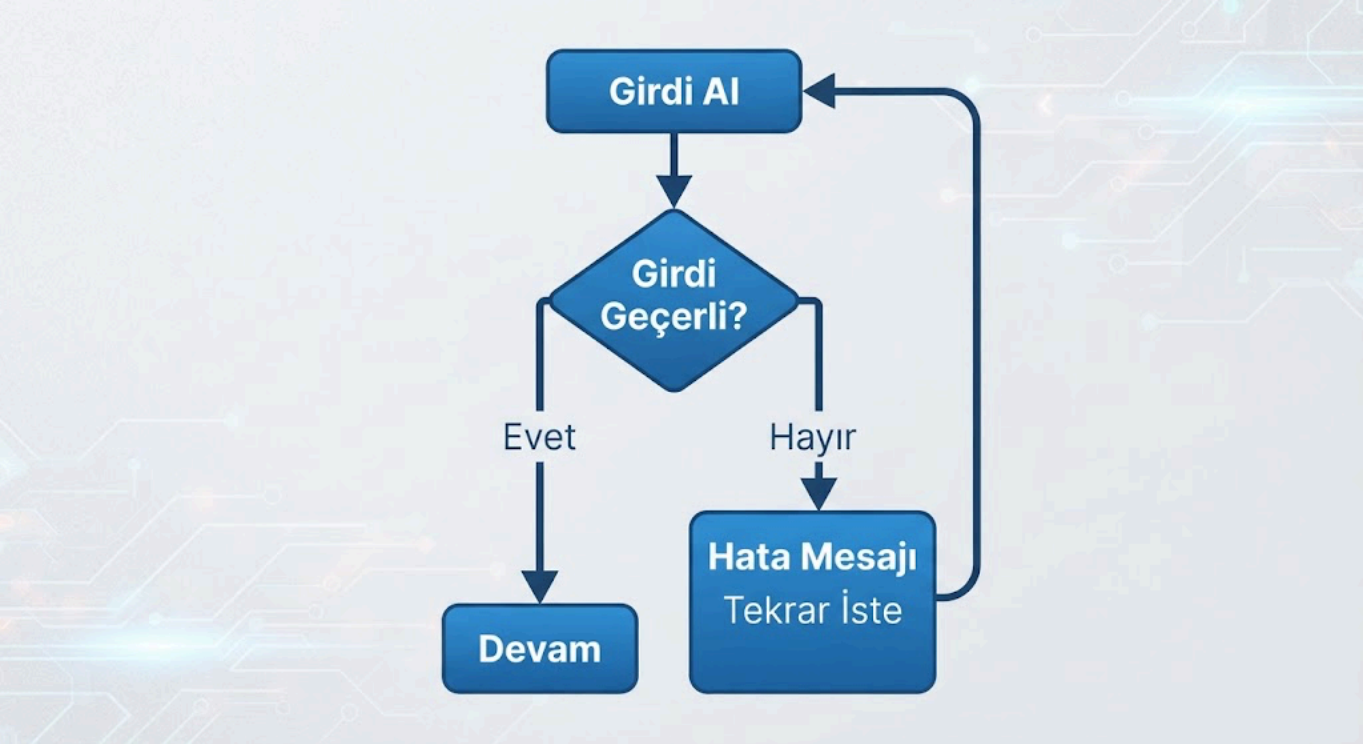
```
printf("Ad:\tAli\n");
printf("Soyad:\tYılmaz\n");
printf("Yaş:\t25\n");
```

**Çıktı:**

```
Ad:     Ali
Soyad:  Yılmaz
Yaş:    25
```

## 4.9 Girdi Doğrulama

Kullanıcıdan alınan verinin geçerli olup olmadığını kontrol etmek gerekir.




**Örnek:**

```c
int yas;
printf("Yaşınızı girin (0-120): ");
scanf("%d", &yas);

while (yas < 0 || yas > 120) {
    printf("Geçersiz yaş! Tekrar girin: ");
    scanf("%d", &yas);
}

printf("Yaşınız: %d\n", yas);
```

---

# 5. UYGULAMA: ÖRNEK PROGRAMLAR

## 5.1 Örnek 1: İki Sayının Toplamı

**Problem:** Kullanıcıdan iki sayı alıp toplamını ekrana yazdıran program.

**C Dili:**

```c
#include <stdio.h>

int main() {
    int sayi1, sayi2, toplam;

    printf("Birinci sayıyı girin: ");
    scanf("%d", &sayi1);
```

```
    printf("İkinci sayıyı girin: ");
    scanf("%d", &sayi2);

    toplam = sayi1 + sayi2;

    printf("Toplam: %d\n", toplam);

    return 0;
}
```

**Çalışma Örneği:**

```
Birinci sayıyı girin: 15
İkinci sayıyı girin: 25
Toplam: 40
```

**Bellek Temsili:**

```
+-------------------+
| sayi1 = 15        |
+-------------------+
| sayi2 = 25        |
+-------------------+
| toplam = 40       |
+-------------------+
```

## 5.2 Örnek 2: Dairenin Alanı

**Problem:** Yarıçapı kullanıcıdan alınan dairenin alanını hesaplayan program.

**Formül:** Alan = π × r²

```
#include <stdio.h>
#define PI 3.14159

int main() {
    float yaricap, alan;

    printf("Yarıçapı girin: ");
    scanf("%f", &yaricap);

    alan = PI * yaricap * yaricap;

    printf("Dairenin alanı: %.2f\n", alan);
```

```
    return 0;
}
```

**Çalışma Örneği:**

```
Yarıçapı girin: 5
Dairenin alanı: 78.54
```

## 5.3 Örnek 3: Ortalama Hesaplama

**Problem:** Üç sınav notunun ortalamasını hesaplayan program.

```c
#include <stdio.h>

int main() {
    float not1, not2, not3, ortalama;

    printf("1. notu girin: ");
    scanf("%f", &not1);

    printf("2. notu girin: ");
    scanf("%f", &not2);

    printf("3. notu girin: ");
    scanf("%f", &not3);

    ortalama = (not1 + not2 + not3) / 3;

    printf("Ortalama: %.2f\n", ortalama);

    if (ortalama >= 60) {
        printf("Durum: Geçti\n");
    } else {
        printf("Durum: Kaldı\n");
    }

    return 0;
}
```

**Çalışma Örneği:**

```
1. notu girin: 75
2. notu girin: 82
3. notu girin: 68
Ortalama: 75.00
Durum: Geçti
```

## 5.4 Örnek 4: Basit Hesap Makinesi

**Problem:** Kullanıcıdan iki sayı ve işlem türü alıp sonucu hesaplayan program.

```c
#include <stdio.h>

int main() {
    float sayi1, sayi2, sonuc;
    char islem;

    printf("Birinci sayıyı girin: ");
    scanf("%f", &sayi1);

    printf("İkinci sayıyı girin: ");
    scanf("%f", &sayi2);

    printf("İşlem türünü girin (+, -, *, /): ");
    scanf(" %c", &islem);

    switch (islem) {
        case '+':
            sonuc = sayi1 + sayi2;
            break;
        case '-':
            sonuc = sayi1 - sayi2;
            break;
        case '*':
            sonuc = sayi1 * sayi2;
            break;
        case '/':
            if (sayi2 != 0) {
                sonuc = sayi1 / sayi2;
            } else {
                printf("Sıfıra bölme hatası!\n");
                return 1;
            }
            break;
        default:
            printf("Geçersiz işlem!\n");
            return 1;
    }

    printf("Sonuç: %.2f\n", sonuc);

    return 0;
}
```

**Çalışma Örneği:**

```
Birinci sayıyı girin: 10
İkinci sayıyı girin: 5
İşlem türünü girin (+, -, *, /): *
Sonuç: 50.00
```

### 5.5 Örnek 5: Yaş Hesaplama

**Problem:** Doğum yılını alıp yaşı hesaplayan program.

```c
#include <stdio.h>

int main() {
    int dogum_yili, mevcut_yil, yas;

    printf("Doğum yılınızı girin: ");
    scanf("%d", &dogum_yili);

    printf("Mevcut yılı girin: ");
    scanf("%d", &mevcut_yil);

    yas = mevcut_yil - dogum_yili;

    printf("Yaşınız: %d\n", yas);

    return 0;
}
```

**Çalışma Örneği:**

```
Doğum yılınızı girin: 2000
Mevcut yılı girin: 2026
Yaşınız: 26
```

---

## 6. DEĞİŞKEN, VERİ TİPİ VE INPUT/OUTPUT İLİŞKİSİ




**Örnek Akış:**

```
1. INPUT:  Kullanıcı "25" girer
2. VERİ TİPİ: int olarak belirlenir
3. DEĞİŞKEN: yas = 25 olarak saklanır
4. İŞLEM: yas = yas + 1 hesaplanır
5. OUTPUT: "Yaşınız: 26" ekrana yazılır
```

---

## 7. ÖZET

- **Değişken**, program çalışırken verilerin saklandığı isimlendirilmiş bellek hücreleridir.
- Değişken isimleri harf ile başlamalı, anlamlı olmalı ve anahtar kelime olmamalıdır.
- **Veri tipi**, bir değişkenin bellekte ne kadar yer kaplayacağını ve hangi türde veri saklayacağını belirler.
- Temel veri tipleri: **int, float, double, char, bool, string**.
- **Input**, programın dışarıdan aldığı verilerdir (klavye, dosya, sensör vb.).
- **Output**, programın ürettiği sonuçlardır (ekran, dosya, yazıcı vb.).
- Farklı dillerde input/output fonksiyonları farklıdır: C'de `scanf/printf`, C++'ta `cin/cout`, Python'da `input/print`.
- Biçimlendirilmiş çıktı için format belirleyiciler (`%d`, `%f`, `%s`) ve kaçış karakterleri (`\n`, `\t`) kullanılır.
- Girdi doğrulama, hatalı veri girişini önlemek için önemlidir.

---

## 8. DEĞERLENDİRME SORULARI

1. Değişken nedir? Değişken tanımlama kurallarını maddeler hâlinde yazınız.
2. Aşağıdaki değişken isimlerinden hangileri geçerlidir? Neden?
    - `ogrenci_adi`, `1sayi`, `toplam@deger`, `_gecici`, `if`, `x2`

3. Veri tipi nedir? Temel veri tiplerini ve boyutlarını yazınız.
4. int, float, double, char ve bool veri tiplerini karşılaştırınız.
5. Otomatik ve zorunlu veri tipi dönüşümlerini örneklerle açıklayınız.
6. Input ve output kavramlarını tanımlayınız. Girdi ve çıktı yöntemlerini yazınız.
7. C, C++, Python ve Java dillerinde input/output nasıl yapılır? Örneklerle gösteriniz.
8. Format belirleyiciler nedir? C dilinde kullanılan beş format belirleyiciyi yazınız.
9. Kaçış karakterleri nedir? Beş örnek veriniz.
10. Kullanıcıdan iki sayı alıp çarpımını ekrana yazdıran programı C dilinde yazınız.
11. Yarıçapı kullanıcıdan alınan kürenin hacmini hesaplayan programı yazınız. (Hacim = $4/3 \times \pi \times r^3$)
12. Girdi doğrulama neden önemlidir? Bir örnekle açıklayınız.

---